

FlashKDA 从 SM80 MMA 迁移到 SM100 是否值得？

结合 Kimi K3 技术报告的完整问题解读、B300 实验路线与系统决策指南

核心结论

不做全面 tcgen05 机械改写；做 guarded ValueSlice；下一步验证 CTA Cluster + TMA multicast。

面向团队内部学习、作业报告和 10 分钟答辩准备

证据范围：Kimi K3 Technical Report arXiv:2607.24653v2；FlashKDA commit 1ce47ea；NVIDIA B300-SXM6 AC / SM103 实验

先读这一页：我们到底在回答什么？

题目不是 “SM100 比 SM80 强多少”

真正的题目是：官方 FlashKDA 的 Tensor Core atom 仍是 SM80 世代 `mma.sync`，在 B300/SM103 上是否值得为它重写 SM100 执行引擎，以及这个成本能否转化为 Kimi K3 `prefill`、并发和 SLO `goodput` 改善。

这份文档把问题分为四层：

层次	要回答的问题	我们的证据
模型层	KDA 为什么是 Kimi K3 核心？69/93 层意味着什么？	Kimi K3 论文架构、KDA 公式和系统章节
算子层	FlashKDA 如何把递推算成 <code>chunkwise GEMM</code> ？	源码、矩阵形状、官方 <code>benchmark</code>
硬件层	<code>tcgen05/TMEM</code> 是否解决了真实瓶颈？	SASS、NCU、Phase-6 <code>microbench</code> 、CHUNK 数值模型
系统层	<code>operator</code> 降时能否变成 <code>TTFT/吞吐/goodput</code> ？	Amdahl 边界、TP/KCP、 <code>prefix cache</code> 、 <code>scheduler</code> 分析

最终答案

SM100 值得利用，但利用 SM100 不等于全面换掉 MMA。实测显示，最自然的 Phase-6 `tcgen05+TMEM` 代换在乐观摊销后仍慢 8.7%；真实首要瓶颈是 TP8 代表形状下的 12 CTA / 148 SM `underfill`。ValueSlice 把 `grid` 扩到 96 CTA，使 `fixed` 和 `packed` 单长序列算子约降时 27%。

目录

先读这一页：我们到底在回答什么？	2
目录	3
第一部分：先理解 Kimi K3 为什么需要 KDA	5
1. Kimi K3 的整体架构	5
1.1 69 层 KDA + 24 层 MLA 是怎么来的	5
2. KDA 是什么：用固定大小状态承载长历史	5
2.1 为什么 CHUNK=16 不是随手写的常数	6
2.2 DPLR 是什么，与 KDA 的关系是什么	6
第二部分：从 KDA 数学到 FlashKDA 矩阵和 GPU 执行	7
3. FlashKDA 在做什么	7
3.1 T、H、D、C、V 分别是什么	7
3.2 代表矩阵规模	7
4. “从 SM80 迁移到 SM100” 究竟指什么	8
第三部分：我们怎么做出决策	9
5. 方法：复现与测量 -> 分析 -> 挑战	9
6. 阶段一：官方 FlashKDA 在 B300 上成立	9
6.1 SASS 回答 “它到底用了什么指令”	10
7. NCU 告诉我们：首要瓶颈是 underfill	10
8. 阶段二分析 A：CHUNK 32/64 为什么不是便宜优化	10
9. 阶段二分析 B：tcgen05 + TMEM 为什么没有自动胜出	12
10. 阶段三挑战：ValueSlice 直接攻击并行度	12

10.1 正收益和反例必须同时展示	13
11. 正确性：不能只看一个 max error	14
第四部分：这与 Kimi K3 系统究竟有什么关系	15
12. 我们的实测正好命中了论文的系统动机	15
13. 从 prefill operator 到 TTFT、并发和 SLO goodput	15
13.1 低并发受益，高并发却可能回退	16
14. 通信、缓存和调度视角	17
14.1 卡内：CTA Cluster + TMA multicast 是下一个具体问题	17
14.2 跨卡：TP 与 KCP 对本地加速的两面性	17
14.3 跨请求：prefix cache 可能比单 kernel 加速更影响 TTFT	17
14.4 调度：SLO goodput 是队列和约束问题	17
第五部分：结论、发布建议与下一步	18
15. 如果我们是 FlashKDA 作者，应该发布什么	18
16. 两天和 15 分钟权限下，为什么这套方法仍然可信	18
17. 如果再有一天 B300，最值得跑什么	18
18. 10 分钟答辩应该如何讲	19
附录 A：名词速查	19
附录 B：证据地图	20
附录 C：参考资料	20

第一部分：先理解 Kimi K3 为什么需要 KDA

1. Kimi K3 的整体架构

Kimi K3 是一个 2.78T 总参数、104.2B 激活参数的 MoE 模型，上下文长度可到 1M token。它在序列、深度和宽度三个维度上分别使用 Hybrid KDA-MLA、Attention Residuals 和 Stable LatentMoE。论文报告相对 Kimi K2 的整体 scaling efficiency 约提升 2.5×。[1, pp.1-3, 10-11]

Kimi K3 简化架构：信息流的三个维度



图 1：基于 Kimi K3 论文 Fig.2 和 Table 1 简化重绘。末尾多出的 1 层 Gated MLA 保证最后一层是全局注意力。

1.1 69 层 KDA + 24 层 MLA 是怎么来的

主干重复 3 层 KDA + 1 层 Gated MLA。23 个 hybrid block 给出 69 层 KDA 和 23 层 MLA；主干末尾再加 1 层 MLA，总数正好是 93 层。这是论文的官方架构图和 Table 1 可以交叉核对的结论。[1, pp.3-4, 11]

重要边界

69/93 是层数比例，不是 wall-time 比例。一个 KDA 层还包含投影、短卷积、norm、gate、MoE 等多个算子；不能用 69/93 去直接放大 FlashKDA kernel 的 27% 收益。

2. KDA 是什么：用固定大小状态承载长历史

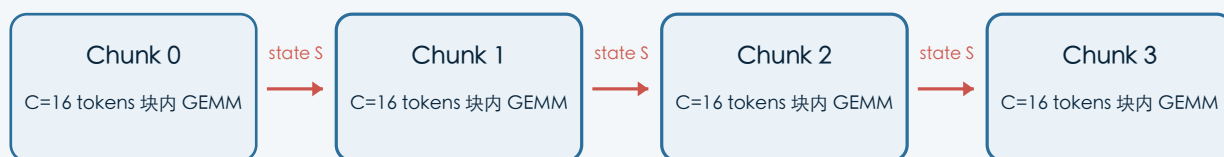
对单个 head，KDA 维护状态 $S_t \in \mathbb{R}^{d_k \times d_v}$ 。每个 token 先通过通道级忘却门 α 衰减旧状态，再用 delta rule 根据 key 和 value 写入新信息：

$$S_t = (I - \beta_t k_t k_t^T) \text{Diag}(\alpha_t) S_{(t-1)} + \beta_t k_t v_t^T$$

$$o_t = S_t^T q_t$$

与 softmax attention 的 KV cache 随 T 增长不同，KDA 的 recurrent state 大小与序列长度无关。这对 1M context 的缓存、迁移和 prefix reuse 非常有吸引力；代价是状态更新有先后依赖，与 GPU 喜欢的宽并行天然冲突。[1, §2.1.1, §5.1]

KDA 的并行性地图：块内并行，块间传状态



不能消失的临界路径：同一 sequence-head 的 $S[t+1]$ 依赖 $S[t]$

论文路线：切 sequence segment

先算段转换，再用 prefix scan / merge 恢复精确入射状态

我们的路线：切 Value 行

各行独立，无 reduction / atomic；代价是公共输入重复请求

图 2：KDA 的并行性不是“有”或“没有”，而是块内、块间和正交维度三种不同问题。

2.1 为什么 CHUNK=16 不是随手写的常数

KDA 的 chunkwise 形式让 chunk 内 token 并行，chunk 间传递 state。Kimi K3 把 log-decay 限制到 $(-5, 0)$ ，所以 16 个 token 累积 log-decay 落在 $(-80, 0)$ ，倒数缩放小于 e^{80} ，仍在 BF16 动态范围内。这让对角 tile 也能使用稠密 Tensor Core GEMM，消除了原先的 position-pair 路径。[1, pp.4-5, Fig.3]

所以 CHUNK=16 同时绑定了三件事：数值范围、局部求解代价、m16n8k16 的自然形状。把 16 改成 32/64 不是一个纯调参问题。

2.2 DPLR 是什么，与 KDA 的关系是什么

DPLR 是 Diagonal-Plus-Low Rank，即把一个大的状态转移矩阵写成 $A = D + U V^T$ 。D 是对角项，处理很便宜； $U V^T$ 是低秩修正，用小量外积捕捉全局耦合。这是很多结构化状态空间模型的常用表达。

KDA 的 token-dependent 转移 $(I - \beta k k^T) \text{Diag}(\alpha)$ 也同时包含通道对角衰减和 rank-1 delta 结构，但它不应被简化成“一个固定 DPLR 矩阵”： α 、 β 、 k 都随 token 变化。说 KDA 比标准 DPLR 高效，通常是指它把 token 并行形式、gate 范围和 Tensor Core 友好的 dense tile 共同设计，而不是说低秩三个字本身就自动快。

第二部分：从 KDA 数学到 FlashKDA 矩阵和 GPU 执行

3. FlashKDA 在做什么

论文对 FlashKDA 的官方定位很清楚：KDA chunkwise 形式在 chunk 内并行、chunk 间串行，朴素执行会在 token-parallel 计算与 state propagation 之间交替，造成 SM 空闲。FlashKDA 用 CUTLASS 给两类阶段分别排程和调优，并且同时服务 training 和 inference prefill。[1, §5.1.1, p.17]

一个容易混淆的缩写

本项目里的 K1 / K2 是 FlashKDA forward 的两个 kernel/stage 标签，其中 K2 指 recurrence 阶段。它不是“Kimi K2 模型”。后文所说“K2 只有 12 CTA”都指 recurrence kernel。

3.1 T、H、D、C、V 分别是什么

符号	含义	本项目的代表值	为什么重要
T	总 token 数 / sequence length	8192	决定 chunk 数：8192/16=512
H	当前 GPU 上的 attention head 数	96 或 TP8 下 12	K2 grid 近似按 sequence×head 展开
D	每 head 的 key/value 工作维度	128	state 近似 128×128；也是 ValueSlice 的可切维
C	chunk / 局部 tile 的 token 数	16	与数值范围、局部求解和 MMA K=16 耦合
V	一个 CTA 负责的 Value 行数	128/64/32/16	D/V 决定每 sequence-head 的 CTA 数

3.2 代表矩阵规模

层次	矩阵 / tensor 形状	直觉
单 token 状态	S: [D,D]=[128,128]	一个 head 的长历史被压进固定大小状态
单 chunk	Q,K,V: [C,D]=[16,128]	16 个 token 形成一个局部并行单元
块内 attention	A: [C,C]=[16,16]	causal lower-triangular 交互
Phase-6 候选	[128,16] @ [16,V] -> [128,V]	V=128 时自然对应 tcgen05 m128n128k16
ValueSlice V16	8 个 [128,16] @ [16,16]	总 Tensor FLOP 不变，CTA 数增加 8×

官方 benchmark 的 T=8192, H=96, D=128 用来复现公开性能。而 Kimi K3 表中有 96 个 attention heads，如果做 TP8，每张卡只剩 H=96/8=12。我们用 H12 不是缩小题目，而是模拟真实 per-GPU 并行度。

4. “从 SM80 迁移到 SM100” 究竟指什么

B300 这张卡的 compute capability 是 10.3，因此我们在构建和证据中更精确地写 SM103 / sm_103a。题目使用“SM100”作为 Blackwell 数据中心新执行模型的总称。“SM80 kernel”也不是说它在 B300 上只用 A100 的全部机制；实际上它已使用 TMA 等新搬运能力，只是 Tensor Core 计算 atom 仍是 `mma.sync.m16n8k16`。

能力	是什么	对 C1 的实际意义
<code>mma.sync</code>	SM80 世代 warp-level Tensor Core MMA，累加器主要在寄存器	官方 FlashKDA 当前 atom，B300 可兼容执行
<code>tcgen05 + TMEM</code>	Blackwell 异步 Tensor Core 路径；累加器放在 Tensor Memory	是直接候选，但要计 <code>alloc</code> 、 <code>descriptor</code> 、 <code>commit/wait</code> 、读回成本
TMA	按 <code>tensor descriptor</code> 在 <code>global/shared</code> 间异步搬运	官方已用；不能把“用 TMA”当作本迁移的新收益
CTA Cluster / DSM	多 CTA 组成 cluster，访问 <code>distributed shared memory</code>	为 ValueSlice 的多 CTA 协作提供基础，但会影响 <code>residency</code> 和同步
TMA multicast	一次 TMA 搬运向 cluster 内多 CTA 广播	可用来减少 ValueSlice 的 <code>q/k/gate</code> 公共输入重复请求
228 vs 164 KiB/SM	B300 实测每 SM 约 228 KiB；A100 约 164 KiB 上限	可放更深 staging，但无法自动修复整卡只有 12 CTA
FP8 / FP6 / FP4	更低精度 Tensor Core 和 block scaling	应先评估规则投影 GEMM； <code>recurrence/state/inverse</code> 需重做长程精度证据

第三部分：我们怎么做出决策

5. 方法：复现与测量 -> 分析 -> 挑战

这个项目的难点不是“把某个 CUDA intrinsic 换掉”，而是先建立决策门：如果官方 baseline 很弱，应先修基础实现；如果 Tensor Core 已饱和，新 MMA 可能值得；如果主要是 HBM 带宽，应先减流量；如果整卡没被喂满，应先找并行度。

阶段	关键问题	方法	出口条件
1. 复现与测量	官方路径和性能是什么？	clean baseline、SASS、NCU、parity	数字可复查，实现路径被证实
2. 分析	候选优化攻击的是真瓶颈吗？	CHUNK 模型、tcgen05 probe、并行性地图	给出 stop/go，保留证据边界
3. 挑战	能否在真瓶颈上做出可交付改变？	ValueSlice + dispatcher + 正反例	正确性、性能、fallback 同时过门

挑战不等于必须把新指令做快

题目允许负结果。如果 microbench 证明继续集成收益有限，用数据停止一条高成本路线，同样是有有效的工程结论。

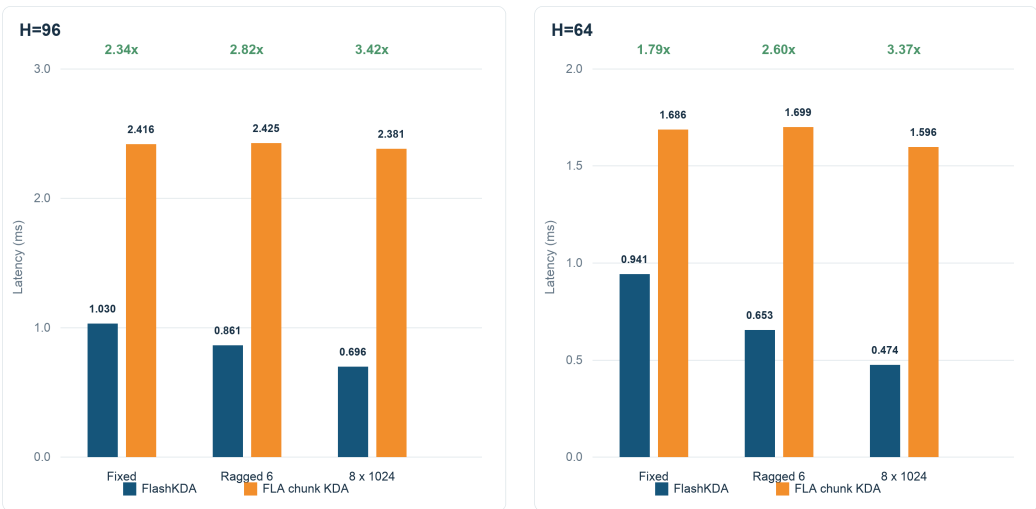
6. 阶段一：官方 FlashKDA 在 B300 上成立

环境：NVIDIA B300 SXM6 AC，CC 10.3，148 SM，每 SM 233,472 B shared memory，L2 约 126.5 MiB；PyTorch 2.10.0+cu130；FlashKDA commit 1ce47ea；CUTLASS 5c149f5；FLA 0.5.2。官方 benchmark 使用 warmup=30、iters=200、repeats=5。

在 H96/H64、fixed/ragged/8×1024 六个 case 中，FlashKDA 相对 FLA chunk KDA 为 1.79-3.42×。这一步说明我们面对的不是一个明显粗糙的 baseline。

Official FlashKDA is already a strong B300 baseline

Forward, T=8192, D=128 | lower latency is better



Source: Slurm Job 17926; official commit 1ce47ea; warmup=30, iters=200, repeats=5.

图 3：官方 FlashKDA 复现。左侧是绝对延迟，右侧是相对 FLA chunk KDA 的 speedup。来源：Job 17926。

6.1 SASS 回答 “它到底用了什么指令”

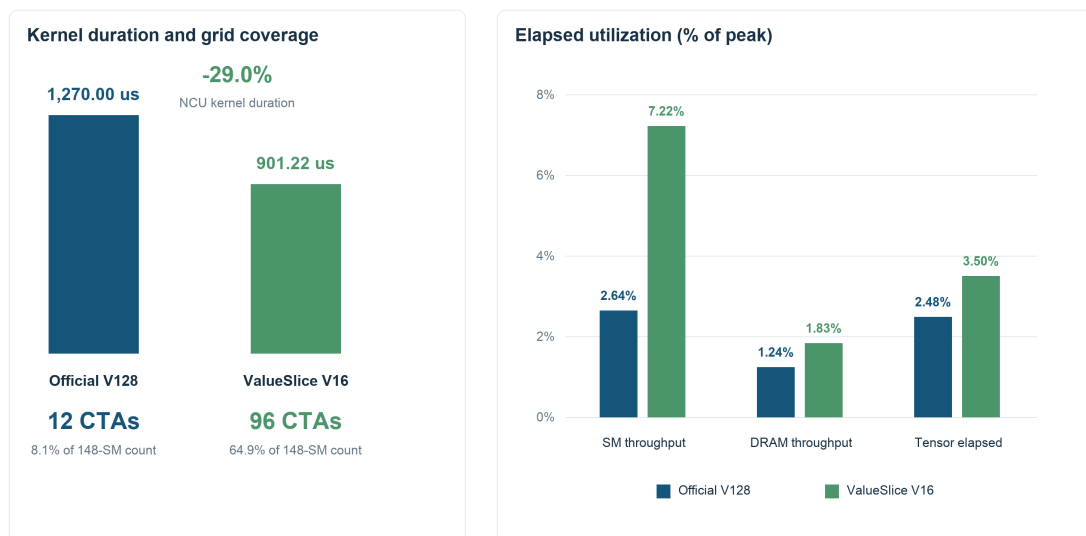
源码选择 SM80_16x8x16_F32BF16BF16F32_TN。recurrence cubin 静态 SASS 中有 3,640 条 HMMA.16816.F32.BF16，而 TCGEN/UTCMMA 为 0。静态条数不等于运行时动态执行次数，但它足以证明题目的起点成立。

7. NCU 告诉我们：首要瓶颈是 underfill

对 TP8 代表形状 T8192/H12/D128，官方 recurrence grid 只有 $1 \times 12 \times 1 = 12$ CTA。B300 有 148 SM，因此即使每个 CTA 都很忙，一波也最多覆盖 8.1% 的 SM。NCU 又显示 SM throughput 2.64%、DRAM throughput 1.24%、tensor pipe elapsed 2.48%。这不像传统的 compute saturation，也不像 HBM bandwidth saturation。

ValueSlice exposes parallelism; B300 remains far from saturation

Targeted NCU on K3 TP8 shape: T=8192, H=12, D=128



Source: Job 17965 NCU summaries. Tensor uses pct_of_peak_sustained_elapsed (not the active-cycle denominator).

图 4：目标形状下的 NCU 瓶颈。ValueSlice V16 将 CTA 从 12 增至 96，同一 profiler 口径的 duration 从 1.27 ms 降至 901.22 us。来源：Job 17965。

如何准确描述这个瓶颈

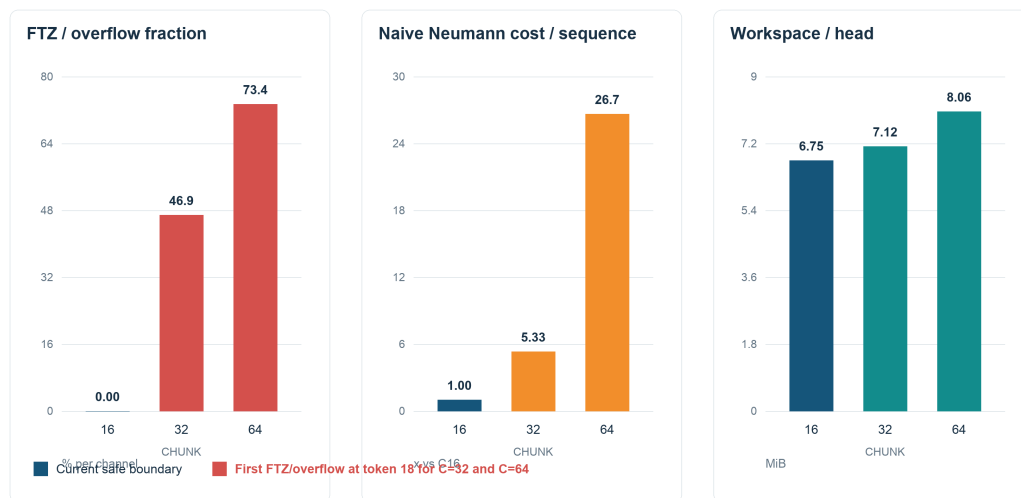
grid underfill + chunk recurrence critical path + CTA 内 TMA/issue latency。不要只说 “occupancy 低”，也不要把 active-cycle tensor 百分比当成整卡利用率。结论来自 grid、duration、SM/DRAM throughput、scheduler 和 timeline 的交叉。

8. 阶段二分析 A：CHUNK 32/64 为什么不是便宜优化

当 lower bound 仍为 -5 且沿用当前无 rescale 的指数恢复方式时，C32 和 C64 在第 18 个 token 就首次出现 FTZ/overflow。同时，把 C16 的朴素 Neumann 密集幂级数直接延伸，总序列计算会增至 5.33x 和 26.67x。workspace 仅从 6.75 MiB/head 增到 7.125/8.063 MiB，所以它反而不是第一个破掉的条件。

Why CHUNK=16 is a numerical and compute boundary

Mechanical C=32/64 enlargement breaks the current unscaled path before workspace becomes dominant



Source: Job 17935 analytical sweep, T=8192, H=12, D=128, lower_bound=-5. FLA block probe is not used as a FlashKDA timing claim.

图 5：CHUNK 增大后的三重边界：数值范围最先失效，朴素局部求解代价随后爆炸，workspace 只是温和上升。来源：Job 17935。

FLA 的 safe/block 设计在小形状上证明，加入 rescale 和 block solve 后 C32/C64 可以做对。但这已经是算法重设，不是将常量 16 改成 64。因此我们对“机械放大 CHUNK”做 NO-GO，而不是宣布所有大 chunk 算法永远无效。

9. 阶段二分析 B：tcgen05 + TMEM 为什么没有自动胜出

我们专门选了对 tcgen05 最有利的 Phase-6：[128, 16]@[16, V]，V=16/32/64/128 都可映射到 m128nVk16。L0 让 mma.sync 和 tcgen05 各自拿到偏好的片上布局，是对新指令乐观的比较；L1 再加入 state/gate 与保守重排，用来暴露集成风险。

对正式 V128、grid=12、inner=64，L0 的 mma_time/tcgen_time=0.920x，即 tcgen05 仍慢约 8.7%。one-shot 时固定协议成本更明显。V16 的纯 core 有形状相关潜力，但当前转换又会吃掉它。

tcgen05 direct-swap gate fails at K3's V=128

Measured MMA/TCGEN speedup; above 1.0 favors tcgen05, below 1.0 favors mma.sync



Source: Job 17937 Phase-6 microbenchmark; warmup=30, iters=200, repeats=5. L0/L1 are probes, not full-K2 timings.

图 6：Phase-6 tcgen05 探针。1.0 以下表示 tcgen05 慢于 mma.sync。最关键的 V128 乐观摊销点仍未跨过 1.0。来源：Jobs 17936/17937。

这个负结果否决了什么

它否决的是保持现有 K2 数据流，只替换一个 Phase-6 MMA。它不能否决未来让 Phase 1/3/4/6 共享转置布局、跨 phase 保持 TMEM-resident 的完整数据流重写。

10. 阶段三挑战：ValueSlice 直接攻击并行度

K2 recurrent state 的 Value 行互相独立。我们将 D=128 按 V=64/32/16 切分，每个 CTA 只更新一组 V×D state rows。因为行之间没有 reduction 和 atomic，每个输出元素内部的运算顺序不变。

V	每 sequence-head CTA 数 D/V	H12 总 CTA	理论一波覆盖 148 SM
128	1	12	8.1%
64	2	24	16.2%
32	4	48	32.4%
16	8	96	64.9%

每 CTA、每 recurrence tile 的 Tensor Core 工作可写为 $6 \cdot C \cdot D \cdot V + 4 \cdot C^2 \cdot V$ 。slice 数是 D/V，两者相乘后总 Tensor FLOP 与 V 无关。因此加速不是少算了，而是用更多独立 CTA 缩短了整卡临界路径。代价是 q/k/gate 等 slice-independent 输入被重复请求。

ValueSlice needs sequence-distribution-aware dispatch

Median latency reduction relative to V128; positive is faster, negative is slower



Source: Job 17947, B300/SM103, H=12, D=128; each cell is the median across 3 repeat medians.

图 7：ValueSlice 的性能取决于 sequence 分布，不只是 total tokens。左侧是延迟，右侧是相对 V128 的变化。来源：Job 17947。

10.1 正收益和反例必须同时展示

输入分布（均 T=8192,H=12）	V128	最佳候选	结果	dispatcher
fixed 1x8192	0.7807 ms	V16 0.5691 ms	-27.1%	auto V16
packed 1x8192	0.7850 ms	V16 0.5740 ms	-26.9%	auto V16
ragged 6	0.3403 ms	V64 0.2810 ms	-17.4%	未标定，回退
8x1024	0.1561 ms	V64 0.1540 ms	-1.3%	低于 guard
32x256	0.1253 ms	V16 0.2585 ms	+106% 更慢	V128

最关键的不是“V16 最快”，而是什么时候 V16 最快。当输入是单个长序列，自然并行度低，切 Value 值得；当有 32 个短序列，负载本身已提供大量 CTA，再切只剩调度与重复流量成本。这就是 guarded dispatcher 而非全局开关的原因。

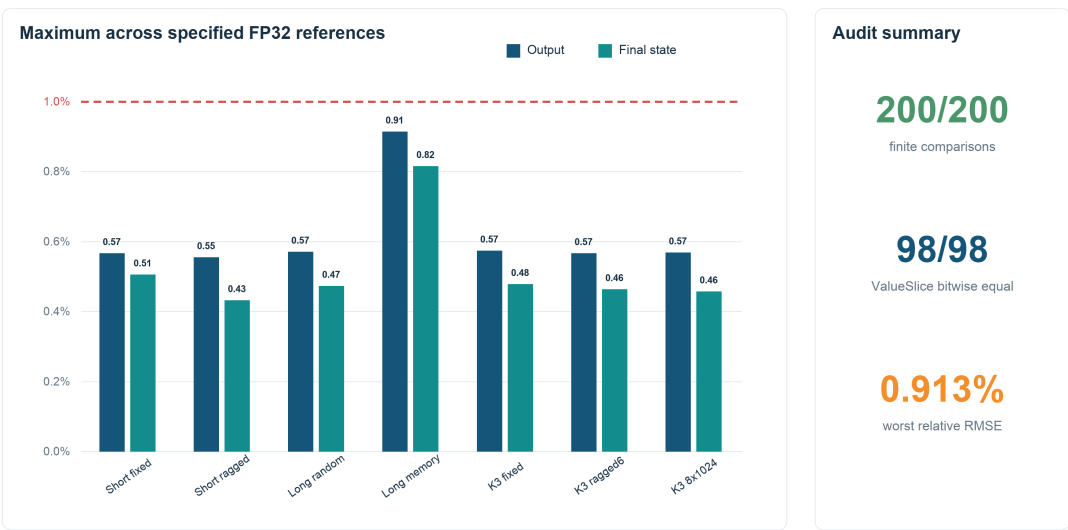
11. 正确性：不能只看一个 max error

验证分为三层：官方 clean extension 对 patched V128 的 parity；V16/V32/V64 对 V128 的 bitwise 对拍；FlashKDA 对题目指定 FLA naive/chunk reference 的长序列、ragged、state carry 和 long-memory gate 对拍。

口径	结果	应该怎么说
所有 comparison rows	200/200 finite	全部为有限数，不等于 200 个独立模型样本
ValueSlice vs V128	98/98 bitwise equal	切行没改变每行归约顺序
长序列参考关系	worst observed relative RMSE 0.9131%	是观测值，不是预注册的模型精度门槛
public FP32 state buffer	与 BF16 路径结果相同	内部仍在 BF16 舍入，不是全 FP32 recurrence 对照

Observed relative RMSE remains below 1%

Worst observed FlashKDA error by workload and tensor; ValueSlice itself is bitwise-equal to V128



Source: Job 17934; FLA 0.5.2; naive.py SHA 60a32285..., chunk.py SHA a15aa6ac...; FLA_FLASH_KDA=0.

图 8：正确性不同层级的口径。红色虚线是 1% 可视化参考，不应被写成未预注册样本的 hard threshold。来源：Job 17934。

低精度的正确开始方式

B300 支持 FP8/FP6/FP4 不等于 KDA state 可以直接降精度。更稳妥的顺序是：先在大而规则的输入/输出投影 GEMM 验证 block scaling；再对 state、gate、inverse 分别设计 scale；最后做长上下文 perplexity 和任务级验证。

第四部分：这与 Kimi K3 系统究竟有什么关系

12. 我们的实测正好命中了论文的系统动机

Kimi K3 论文在 §5.1.1 明确写道：Tensor Parallelism 会把 head 分到多张卡，却不会缩短 recurrence；因此在纯 TP 部署中，超长序列 prefill 会因每 rank 只剩少数 head 而使大量 SM 空闲。[1, pp.17-18]

我们的 H12 实测就是这句话的硬件化证据：96 heads / TP8 = 12 heads/GPU，官方 recurrence 只有 12 CTA 对 148 SM。这说明 ValueSlice 不是与论文无关的小技巧，而是对官方已识别系统矛盾的一种补充解法。

方法	切分维度	优点	代价 / 边界
论文的 intra-device CP planner	sequence segment	超长 T 时可沿序列展开；精确合并入射状态	需计算段 transition 并 merge；实现复杂
论文的 KCP	cross-device sequence segment	只交换固定大小 transition/state fragment；不随 T 增长	all-gather 和 prefix scan；改变多卡调度
我们的 ValueSlice	value rows	行独立，无 reduction/atomic；补充 H 过少	最多提供 D/V 倍 CTA；公共输入重复

这三条路线不是互斥的。在极长 T 上，sequence segmentation 可提供更多并行度；在 per-GPU head 很少且 D=128 时，ValueSlice 是简洁的补充；未来 dispatcher 应基于 T、H、nseq、SM 数和缓存成本选择组合，而不是只有一个永久 kernel。

13. 从 prefill operator 到 TTFT、并发和 SLO goodput

从 27% operator 降时到 SLO goodput：中间有五道门



一阶 Amdahl 映射

若 FlashKDA forward 占整个 prefill 时间 p ，则 prefill 降时约 $0.27p$ ；它仍不等于 SLO goodput 增益。

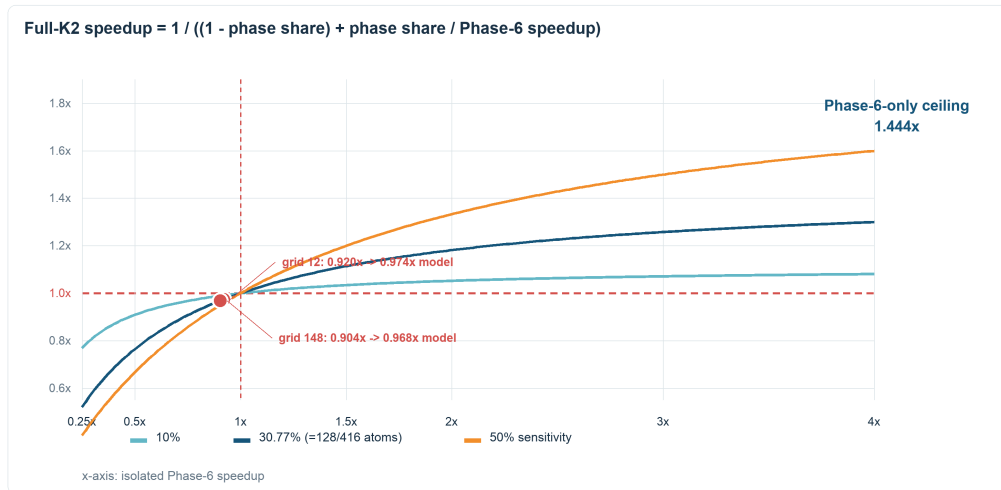
图 9：算子数字只是系统因果链的起点。

令 p 为 FlashKDA forward 占整个 prefill wall time 的比例，实测 operator 降时 $r=0.27$ 。忽略 overlap 变化时，prefill 降时约为 $p*r$ ，理想容量加速为 $1/(1-p*r)$ 。

假设 p	预计 prefill 降时	理想容量加速	应怎么解读
20%	5.4%	1.057×	FlashKDA 只是整个 prefill 的小部分
40%	10.8%	1.121×	有意义，但仍远小于 27%
60%	16.2%	1.193×	只在 KDA forward 占很大时才接近两位数

Amdahl sensitivity: a Phase-6 win has a bounded full-K2 impact

Analytical model only; measured V128 direct-swap points are slower than 1.0x



Model source: exact static HMM atom count (Phase 6 = 128, K2 total = 416). Markers use Job 17937 V128 L0 inner=64 measurements.

图 10：Amdahl 敏感性，用于说明端到端上限，不是 Kimi K3 实测结果。

13.1 低并发受益，高并发却可能回退

ValueSlice 解决的是自然并行度不足。单个长 prompt 没有足够 sequence/head 可以喂满 GPU，所以切 Value 有效。continuous batching 中如果已有很多短 sequence，新 slice 会与其他请求争抢 CTA 槽位和缓存，而且重复搬运。32×256 的 +106% 反例说明它可以伤害吞吐。

所以 dispatcher 必须看 workload：fixed 或 packed nseq=1、T 足够长、H/SM 较低、BF16 state 且收益超过 guard band 时启用；未建模 varlen 和高自然并行度回退 V128。

14. 通信、缓存和调度视角

14.1 卡内：CTA Cluster + TMA multicast 是下一个具体问题

ValueSlice V16 让同一个 head 有 8 个 CTA，而 q/k/gate 等 common inputs 会被 8 份 slice 重复请求。纸面 source-request 模型在 T4096 单 head 上给出：V16 总请求 29.188 MiB，其中理想 8-CTA multicast 可消除 23.734 MiB，约 81.3%。但这不是实测 HBM bytes，也没计 cluster residency、barrier 和 DSM 压力。

因此下一轮不应写“我们将使用 cluster 进一步加速”，而应写：对 2/4/8-CTA cluster 实测 active clusters/SM、multicast bytes、sync stalls、L2/DRAM bytes 和完整 K2 duration，只有净收益过门才合入。

14.2 跨卡：TP 与 KCP 对本地加速的两面性

TP8 一方面把 H96 变成 H12，造成我们优化的 underfill；另一方面它引入 NCCL collective。ValueSlice 没有跨 GPU 通信，也不改变 collective 消息体。本地 K2 节省约 0.21 ms 是否出现在端到端关键路径上，取决于它与 projection/GEMM/NCCL 的 overlap。

论文的 KDA Context Parallelism 则利用 KDA state 固定大小这个优势，每 rank 先计算本地段的 cumulative transition 和 zero-state contribution，再用一次 fixed-size all-gather 交换 fragment，通过 associative prefix composition 恢复每段入射状态。这与 softmax CP 需交换随 T 增长的 KV block 有本质差别。[1, §5.1.2, p.18]

14.3 跨请求：prefix cache 可能比单 kernel 加速更影响 TTFT

Kimi K3 是 KDA-MLA hybrid：MLA KV cache 随 token 增长，KDA state 是每 request 一份固定大小状态。一个 prefix 只有在 MLA KV 和所有 KDA state checkpoint 于同一 token 边界都可恢复时才可复用。论文因此将两类 cache 放进同一 paged pool，并把细粒度 hash block 与粗粒度 physical block 解耦。[1, §5.4.1, pp.22-24]

论文给出的生产例子是：1M context 场景中，编码请求可能带有 400K token 已有 prefix，但只需要新 prefill 4K token。一次 cache hit 避免重算 400K，其影响可以比把 4K 中某个 operator 再优化 27% 大得多。这不是说 kernel 不重要，而是说 kernel、cache 和 scheduler 必须放在同一个收益模型里。

14.4 调度：SLO goodput 是队列和约束问题

论文的 fleet-level 设计包括 cache-aware affinity scheduling 和 budget-based admission control。前者优先把会话路由到拥有其 prefix cache 的 cluster；后者为不同长度请求类型分配独立资源预算，避免 1M-token 突发拖垮短请求 TTFT。[1, §5.4.3, p.25]

SLO goodput 指单位时间内满足延迟 SLO 的有效请求数，它不等于裸 tokens/s。一个 kernel 加速可能降低服务时间、减少排队并提高 goodput，也可能被 NCCL overlap 或 cache miss 完全掩盖。没有真实 prompt distribution、continuous batching 和 SLO sweep，我们就不应给出一个伪精确的 goodput 百分比。

第五部分：结论、发布建议与下一步

15. 如果我们是 FlashKDA 作者，应该发布什么

路线	当前决策	证据	重新打开的门槛
全面 mma.sync -> tcgen05	NO-GO	Phase-6 V128 乐观摊销仍 0.920×	跨 Phase 1/3/4/6 共享布局与 TMEM 生命周期，完整 K2 净收益过门
CHUNK 32/64 机械放大	NO-GO	token 18 开始 FTZ/overflow；朴素代价 5.33×/26.67×	加入 rescale/block solve，重新证明数值和总代价
guarded ValueSlice	GO	12 -> 96 CTA；fixed/packed-one 约 -27%	仅在已标定低并发长 prefill 域启用，保留 V128 fallback
CTA Cluster + TMA multicast	NEXT	直接针对 common-input 重复请求	实测 residency、bytes、sync stalls 和完整 K2 duration

推荐发布形式

V128 mma.sync fallback + B300/SM103 已标定 ValueSlice + 架构/形状 guard + 现场可禁用开关。
packed nseq=1 按 fixed B1 定标；未建模 nseq>1 varlen 回退。CUDA 13 / sm_103a 单独构建与 CI，持续覆盖 bitwise、reference、CUDA Graph 和反例。

16. 两天和 15 分钟权限下，为什么这套方法仍然可信

任务	Job	实测 GPU wall time	证据目标
官方 clean baseline	17926	93 s	强基线与公开形状
official vs patched V128 parity	17929	6 s	隔离 baseline 污染
naive/chunk reference	17934	33 s	正确性矩阵
CHUNK 16/32/64	17935	41 s	数值/计算/workspace
tcgen05 Phase-6	17937	16 s	新指令 stop gate
ValueSlice + dispatcher	17947	10 s	正例、反例和 auto policy
targeted NCU	17965	15 s	underfill 因果链

七个最终 job 的实测 GPU wall time 合计约 214 s。权限限制改变了实验组织：我们把编译、正确性、性能和 profiler 拆成自包含 job，让负结果可以快速停止下游高成本实现。它没有降低单卡 kernel 因果链的标准。

但资源限制确实留下了明确的空白：没有 TP8 NCCL、完整 Kimi K3 checkpoint、真实线上 trace、TTFT/TPOT/SLO sweep 和跨机置信区间。高质量结论不是隐藏这些空白，而是把“实测”、“纸面模型”和“系统推断”严格分开。

17. 如果再有一天 B300，最值得跑什么

优先级	实验	核心指标	什么结果改变决策
P0	2/4/8-CTA cluster + TMA multicast microbench	cluster residency、L2/DRAM bytes、barrier stalls、duration	完整 K2 净收益 >3% 且不伤高并发
P0	ValueSlice 与 intra-device CP 组合 sweep	T、H、nseq、V、planner 选择	找到两种并行分解的明确分界线
P1	完整 K2 共享 TMEM 生命周期原型	完整 duration、TMEM/TMA、register/shared usage	只有完整路径跨过 V128 baseline 才重开 tcgen05
P1	单机模拟 serving workload mix	fixed/packed/ragged 比例、p50/p95、throughput	将 dispatcher 从手工 guard 升级为 workload-weighted policy

18. 10 分钟答辩应该如何讲

时间	要讲的一句话	证据
0:00-1:00	问题是否值得重写，不是新硬件功能列表	四项 worthiness gate
1:00-2:00	Kimi K3 为什么依赖 KDA，TP8 为什么得到 H12	93 层、69 KDA、96 heads
2:00-3:30	官方很强，但编译后确实是 HMMA 而非 tcgen05	1.79-3.42×；3640 HMMA
3:30-5:00	真实瓶颈是 12 CTA / 148 SM underfill	NCU SM/DRAM 2.64%/1.24%
5:00-6:20	tcgen05 direct swap 和机械大 CHUNK 都被数据否决	0.920×；token 18；5.33×/26.67×
6:20-8:20	ValueSlice 不改 FLOP 而扩 CTA，同时给出正例与反例	12->96；-27%；32×256 +106%
8:20-9:20	27% 只是 operator，系统收益受 p、NCCL、cache、scheduler 约束	Amdahl 表 + 论文 §5
9:20-10:00	发布 hybrid，下一步 cluster/multicast	fallback + guarded policy + NEXT gate

答辩的收尾句建议固定为：

收尾句

SM100 的价值在于给我们更多解决真瓶颈的机制，不在于迫使所有旧 MMA 都换名字。对当前 FlashKDA，证据支持保留 `mma.sync`、发布 guarded ValueSlice，再用 cluster/multicast 解决新并行度带来的重复流量。

附录 A：名词速查

名词	一句话解释
prefill	对 prompt 的所有 token 并行建立中间状态/缓存，主要影响 TTFT。
decode	逐 token 生成，主要影响 TPOT；Kimi K3 论文使用独立 fused KDA decode/replay 路径。
TTFT	Time To First Token，从请求到首 token 的时间。
TPOT	Time Per Output Token，decode 期间相邻输出 token 的平均间隔。
SLO goodput	单位时间内满足 TTFT/TPOT 等 SLO 的有效请求或 token 量。
TP	Tensor Parallelism，将权重/特征/头分到多 GPU；可降低每卡 H，却不缩短 recurrence。
CP / KCP	Context Parallelism；KCP 用 KDA 段 transition 和 state fragment 做 fixed-size 跨卡组合。
CTA	Cooperative Thread Array，也就是 CUDA block；grid 中 CTA 数决定整卡有多少独立工作。
underfill	独立 CTA 数显著少于 SM 数，大量 SM 根本没有 block 可执行。
occupancy	活跃 warp 相对硬件上限的比例；它是诊断指标，不是单独因果。
TMA	按 tensor descriptor 异步搬运的硬件引擎。
TMEM	Blackwell Tensor Memory，为 tcgen05 累加器和数据流提供专用片上存储。
DSM	Distributed Shared Memory，cluster 内 CTA 可访问对方 shared memory 的机制。
multicast	一次 TMA 搬运同时喂给 cluster 内多个 CTA。
SASS	GPU 机器指令；用于证明编译后真正出现 HMMA 还是 UTC/TCGEN。
NCU	Nsight Compute，用于获取 kernel duration、grid、throughput、scheduler 等指标。
DPLR	Diagonal-Plus-Low Rank， $A=D+UV^T$ 的结构化状态转移表达。
dispatcher	根据架构、形状、序列分布和数据类型选择 kernel 变体的运行时策略。

附录 B：证据地图

要证明的事	主证据	原始产物
Kimi K3 的 93 层 hybrid 架构	Kimi K3 Technical Report Fig.2 / Table 1	2607.24653v2.pdf pp.3-5, 11
纯 TP 导致每 rank head 少、长 recurrence 让 SM 空闲	Kimi K3 §5.1.1	2607.24653v2.pdf pp.17-18
官方 FlashKDA 是强基线	Job 17926	01_official_benchmark_17926.log
当前 atom 是 SM80 MMA	source + SASS	sass_opcode_summary.csv
H12 主要是 underfill	Job 17965 NCU	05_targeted_ncu_summary/metrics_17965.csv
tcgen05 direct swap 信号不足	Jobs 17936/17937	03_tcgen05_probe_17937.csv + SASS
CHUNK 32/64 不能机械放大	Job 17935	04_chunk_analysis_17935.csv
ValueSlice 在已标定域收益约 27%	Job 17947	05_dispatch_upgrade_17947.csv
正确性和口径边界	Jobs 17929/17934	baseline_parity.json + reference_correctness CSV
prefix cache / KCP / serving 映射	Kimi K3 §5.1.2, §5.4	2607.24653v2.pdf pp.18, 22-25

附录 C：参考资料

[1] Kimi Team. Kimi K3: Open Frontier Intelligence. arXiv:2607.24653v2, 7 Aug 2026. 本文主要使用 pp.3-5（架构与 KDA）、p.11（Table 1）、pp.17-18（KDA kernels/KCP）、pp.22-25（prefix cache、decode kernel 和 fleet scheduling）。

[2] MoonshotAI. FlashKDA: Flash Kimi Delta Attention. GitHub repository. 本项目固定 baseline commit: 1ce47ea3bb22c84eb9cc665028399cf35e8ffb0b；CUTLASS pin: 5c149f5。

[3] Assignment 02 Kimi KDA Team. FlashKDA 官方 Kernel 从 SM80 MMA 迁移到 SM100 是否值得？B300 实验包和最终报告，2026-09-03。

[4] NVIDIA. CUDA C++ Programming Guide: Compute Capabilities; CUTLASS CuTe DSL tcgen05 API documentation. 用于 SM103、cluster、TMA、TMEM 与 tcgen05 机制解释。

[5] FLA 0.5.2 KDA naive/chunk reference implementation. 本项目验证时设置 FLA_FLASH_KDA=0，避免参考实现回落到待测 FlashKDA。

文档使用提醒

本文是团队消化和决策指南，不替代原论文和原始实验日志。答辩中所有百分比必须保留对象和口径：operator / prefill / TTFT / TPOT / goodput 不得互换。